COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURES AND ALGORITHM	**SECTION:**	CPE21S1
DATE PERFORMED:	3/8/2026	**DATE SUBMITTED:**	10/8/2026
NAME:	LOREEN JAE D. PILAPIL	**INSTRUCTOR:**	ENGR. QUEJADO

ACTIVITY NO. 5 **[ACTIVITY NAME]**

1. PROCEDURE OUTPUT

ILO A:

CODE

main.cpp > main()

```cpp
#include <iostream>
#include <queue>
#include <string>

void display(std::queue<std::string> q)
{
    while (!q.empty())
    {

        std::cout << q.front() << " ";
        q.pop();
    }
    std::cout << std::endl;
}

int main()
{

    std::queue<std::string> students;

    std::string names[] = {"Loreen", "Keara", "Guillian", "Jasmine", "Marion"};

    for (int i = 0; i < 5; i++)
    {
        students.push(names[i]);
    }


    std::cout << "The queue is:  ";
    display(students);

    std::cout << "empty? "
              << students.empty() << std::endl;


    std::cout << "size: "
              << students.size() << std::endl;

    std::cout << "Front: "
              << students.front() << std::endl;

    std::cout << "Back: "
              << students.back() << std::endl;


    students.pop();
    std::cout << "pop: ";
    display(students);

    students.push("Sidney");
    std::cout << "push: ";
    display(students);



    return 0;
}
```

OUTPUT

```
The queue is:  Loreen Keara Guillian Jasmine Marion
empty? 0
size: 5
Front: Loreen
Back: Marion
pop: Keara Guillian Jasmine Marion
push: Keara Guillian Jasmine Marion Sidney
```

ANALYSIS

ILO A successfully demonstrated how to use a queue using the C++ STL. The activity showed how operations such as push, pop, front, back, size, and empty work on a queue.

ILO B:

CODE

```cpp
#include <iostream>
#include "QUEUE.h"

int main(){


    qnode<char>* front = nullptr;
    qnode<char>* back = nullptr;


    enqueue(&front, &back, 'L');
    std::cout << front->data << " " << back->data << std::endl;
    enqueue(&front, &back, 'O');
    std::cout << front->data << " " << back->data << std::endl;
    enqueue(&front, &back, 'R');
    std::cout << front->data << " " << back->data << std::endl;
    enqueue(&front, &back, 'E');
    std::cout << front->data << " " << back->data << std::endl;
    enqueue(&front, &back, 'E');
    std::cout << front->data << " " << back->data << std::endl;
    enqueue(&front, &back, 'N');
    std::cout << front->data << " " << back->data << std::endl;
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    frontvar(&front);
    Display(&front, &back);




    return 0;
}
```

```cpp
#ifndef QUEUE_H
#define QUEUE_H


template <typename T>
class qnode{
    public:
        T data;
        qnode *next;

};


template <typename T>
qnode<T> *new_Node(T newData){
    qnode<T>* newNode = new qnode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}



template <typename T>
void enqueue(qnode<T>** frontptr, qnode<T>** backptr, T newData){
    qnode<T>* newNode = new_Node(newData);

    if ((*frontptr) == nullptr && (*backptr) == nullptr){

        (*frontptr) = newNode;
        (*backptr) = newNode;
    }



    (*backptr)->next = newNode;
    (*backptr) = newNode;


}



template <typename T>
void dequeue(qnode<T>** frontptr, qnode<T>** backptr){
    qnode<T>* deletenode = nullptr;
    deletenode = (*frontptr);




    if ((*frontptr)->next == nullptr && (*backptr)->next == nullptr){
        (*frontptr) = nullptr;
        (*backptr) = nullptr;
        delete deletenode;
        return;
    }



    (*frontptr) = deletenode->next;
    deletenode->next = nullptr;
    delete deletenode;
}
```

```
template <typename T>
void Display(qnode<T>** frontptr, qnode<T>** backptr)
{

    qnode<T>*  current = (*frontptr);

    while (current !=nullptr)
    {
        std::cout << current->data << " ";
        current = current->next;
    }


    std::cout << std::endl;
}


template <typename T>
void frontvar(qnode<T>** frontptr)
{

    if ((*frontptr) != nullptr)
    {

    }
}
#endif
```

OUTPUT

```
L L
L O
L R
L E
L E
L N
O N
O R E E N
```

ANALYSIS

ILO B demonstrated how to use a queue using linked lists in C++. I learned how enqueue adds elements to the back and dequeue removes elements from the front following the FIFO principle.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues
Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done.

Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.
You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic

COMPUTER ENGINEERING
QUEZON CITY

strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.
5. Defend your choice of internal representation: Why did you use arrays or linked lists?

Output Analysis:
- Provide the output after performing each task above.
- Include your analysis: focus on why you think the output is the way it is.

STEP 1

```cpp
//STEP 1
class Job {
public:
    int id;
    std::string name;
    int pages;

    Job(int i, std::string n, int p)
        : id(i), name(n), pages(p) {}
};
```

STEP 2

```cpp
//STEP 2
template <typename T>
class Node {
public:
    T data;
    Node<T>* next;

    Node(const T& job) : data(job), next(nullptr) {}
};
```

STEP 3

```cpp
//STEP 3
template <typename T>
class Printer {
private:
    Node<T>* front;
    Node<T>* back;


public:
    Printer() : front(nullptr), back(nullptr) {}

    ~Printer() {
        while (front != nullptr) {
            Node<T>* temp = front;
            front = front->next;
            delete temp;
        }
    }


    Printer(const Printer&) = delete;
    Printer& operator=(const Printer&) = delete;

    void add(const T& job) {
        Node<T>* newNode = new Node<T>(job);

        if (front == nullptr) {
            front = newNode;
            back = newNode;
        }
        else {
            back->next = newNode;
            back = newNode;
        }


        std::cout << "Job " << job.id << " added." << std::endl;
    }
```

```
    void process() {
        while (front != nullptr) {
            Node<T>* temp = front;


            std::cout << "Processing Job " << temp->data.id
                      << " = User: " << temp-> data.name
                      << " = Pages: " << temp-> data.pages
                      << std::endl;

            front = front->next;


            if (front ==nullptr) {
                back =nullptr;
            }

            delete temp;
        }
    }

};

#endif
```

STEP 4

```
#include "Supplementary.h"
#include <iostream>

int main() {

    Printer<Job> printer;

    Job job1(1, "Loreen", 7);
    Job job2(2, "Marion", 10);
    Job job3(3, "Guillian",3);


    printer.add(job1);
    printer.add(job2);
    printer.add(job3);
    printer.process();

    return 0;
}
```

```
Job 1 added.
Job 2 added.
Job 3 added.
Processing Job 1 = User: Loreen = Pages: 7
Processing Job 2 = User: Marion = Pages: 10
Processing Job 3 = User: Guillian = Pages: 3
```

STEP 5

I chose to use Linked list because it fits the task very well since I need it to use the FIFO principle and I don't need to set a specific number of jobs, it just adds automatically.

Conclusion:

In this Hands-on activity I learned how to use the queue in C++ STL and Linked list. The most challenging part was figuring out how to implement a linked list in the supplementary activity because I understood the STL version better. Overall, I succeeded after several trials and debugging my code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.